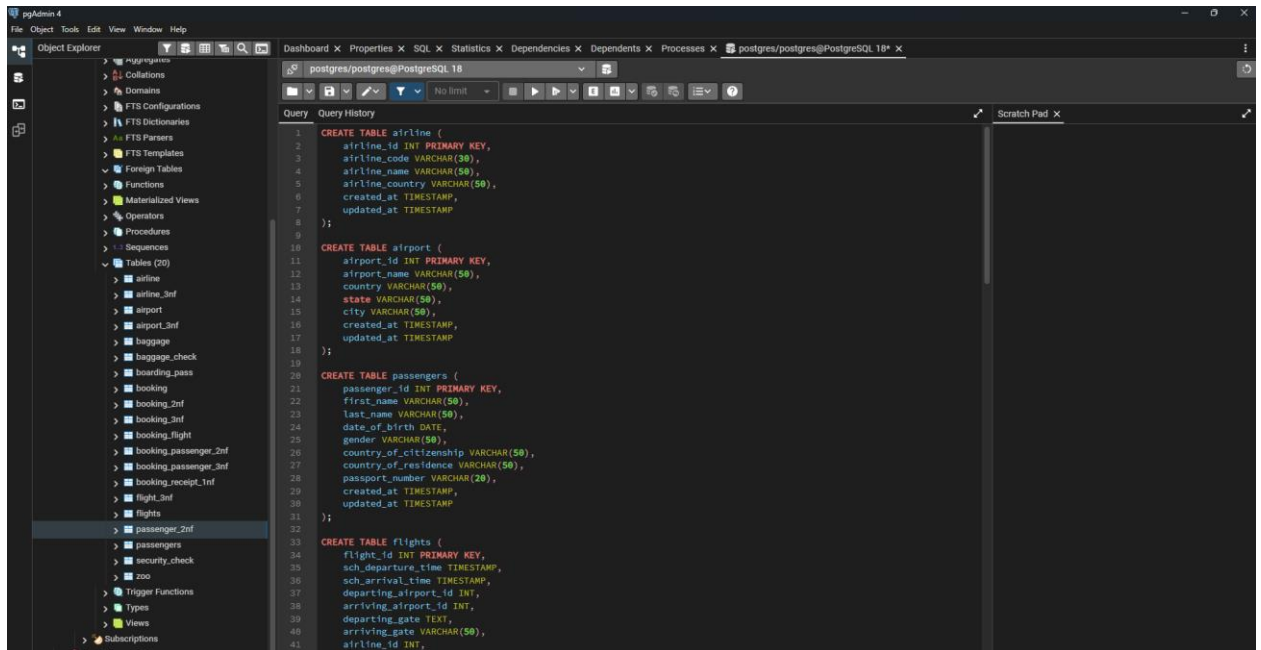


DML

1. Generate and insert 200 random rows in your airport database.
2. Add a new airline named "KazAir" based in "Kazakhstan" to the airline table.
3. Update the airline country "KazAir" to "Turkey".
4. Add three airlines at once: "AirEasy" in "France", "FlyHigh" in "Brazil" and "FlyFly" in "Poland".
5. Delete all flights whose arrival in 2024 year.
6. Increase the price of all tickets in booking table for flights by 15%.
7. Delete all tickets where price is less than 10000.
8. Set a default code 'UNK' (Unknown) for any airline in the airline table where airline_code is missing (NULL).
9. Delete all baggage check records created before June 1, 2023, whose result is still 'Not checked'.
10. Delete all entries from the airport table where state is missing (NULL) and the city is either 'Mlawe' or 'Kepuh'.
11. Insert a new unchecked baggage inspection record and immediately return its newly generated baggage_check_id and created_at timestamp.
12. Convert all country names in the airline table to uppercase to ensure data consistency.
13. Update both the name and country for the airline with airline_id = 5. Set the name to 'Global Airways' and the country to 'United Kingdom', while updating update_at to the current date.
14. Set the state field to 'Capital District' for airports located in specific cities: 'Astana', 'London', and 'Tokyo'.
15. Mark all baggage check records (baggage_check) created in March 2024 with a status of 'Not checked' as completed ('Checked').

First, I created a new tables as to not conflict with other tables from previous lab work – because I divided them into normalization steps, most of them don't contain all sufficient data:



```
47 FOREIGN KEY (departing_airport_id)
48 REFERENCES airport(airport_id),
49
50 FOREIGN KEY (arriving_airport_id)
51 REFERENCES airport(airport_id),
52
53 FOREIGN KEY (airline_id)
54 REFERENCES airline(airline_id)
55 );
56
57 CREATE TABLE booking (
58     booking_id INT PRIMARY KEY,
59     flight_id INT,
60     passenger_id INT,
61     booking_platform VARCHAR(50),
62     created_at TIMESTAMPTZ,
63     updated_at TIMESTAMPTZ,
64     status VARCHAR(50),
65     ticket_price DECIMAL(7,2),
66
67     FOREIGN KEY (flight_id)
68     REFERENCES flights(flight_id),
69
70     FOREIGN KEY (passenger_id)
71     REFERENCES passengers(passenger_id)
72 );
73
74 CREATE TABLE booking_flight (
75     booking_flight_id INT PRIMARY KEY,
76     booking_id INT,
77     flight_id INT,
78     created_at TIMESTAMPTZ,
79     updated_at TIMESTAMPTZ,
80
81     FOREIGN KEY (booking_id)
82     REFERENCES booking(booking_id),
83
84     FOREIGN KEY (flight_id)
85     REFERENCES flights(flight_id)
86 );
87
```

```

88 CREATE TABLE baggage (
89     baggage_id INT PRIMARY KEY,
90     weight_in_kg DECIMAL(4,2),
91     created_at TIMESTAMP,
92     updated_at TIMESTAMP,
93     booking_id INT,
94
95     FOREIGN KEY (booking_id)
96         REFERENCES booking(booking_id)
97 );
98
99 CREATE TABLE baggage_check (
100     baggage_check_id INT PRIMARY KEY,
101     check_result VARCHAR(50),
102     created_at TIMESTAMP,
103     updated_at TIMESTAMP,
104     booking_id INT,
105     passenger_id INT,
106
107     FOREIGN KEY (booking_id)
108         REFERENCES booking(booking_id),
109
110     FOREIGN KEY (passenger_id)
111         REFERENCES passengers(passenger_id)
112 );
113
114 CREATE TABLE boarding_pass (
115     boarding_pass_id INT PRIMARY KEY,
116     booking_id INT,
117     seat VARCHAR(50),
118     boarding_time TIMESTAMP,
119     created_at TIMESTAMP,
120     updated_at TIMESTAMP,
121
122     FOREIGN KEY (booking_id)
123         REFERENCES booking(booking_id)
124 );
125
126 CREATE TABLE security_check (
127     security_check_id INT PRIMARY KEY,
128     check_result VARCHAR(20),

```

Only after that I went ahead with tasks

1. Generate and insert 200 random rows in your airport database.

Query

Query History

1

INSERT INTO airport (airport_id, airport_name, country, state, city, created_at, updated_at)

2

SELECT

3

5000 + gs,

4

'Airport ' || gs,

5

CASE WHEN random() < 0.5 THEN 'Kazakhstan' ELSE 'France' END,

6

NULL,

7

CASE WHEN random() < 0.5 THEN 'Almaty' ELSE 'Paris' END,

8

CURRENT_TIMESTAMP,

9

CURRENT_TIMESTAMP

10

FROM generate_series(1, 200) AS gs;

Data Output

Messages

Notifications

I used CASE WHEN to randomize the selection between France and Kazakhstan as a choose.

2. Add a new airline named "KazAir" based in "Kazakhstan" to the airline table:

Query

Query History

1

INSERT INTO airline (airline_id, airline_code, airline_name, airline_country, created_at, updated_at)

2

VALUES (101, NULL, 'KazAir', 'Kazakhstan', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 1

	airline_id [PK] integer	airline_code character varying (30)	airline_name character varying (50)	airline_country character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone
1	101	[null]	KazAir	Kazakhstan	2026-09-27 17:08:48.291306	2026-09-27 17:08:48.291306

3. Update the airline country "KazAir" to "Turkey"(I used UPDATE, SET, WHERE)

Query

Query History

1

UPDATE airline

2

SET airline_country = 'Turkey', updated_at = CURRENT_TIMESTAMP

3

WHERE airline_name = 'KazAir';

Data Output

Messages

Notifications

≡

+

📄

▼

📋

▼

🗑️

🔍

📥

📶

SQL

Show

	airline_id [PK] integer	airline_code character varying (30)	airline_name character varying (50)	airline_country character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone
1	101	[null]	KazAir	Turkey	2026-09-27 17:08:48.291306	2026-09-27 17:09:49.936158

4. Add three airlines at once: "AirEasy" in "France", "FlyHigh" in "Brazil" and "FlyFly" in "Poland":

Query

Query History

1

INSERT INTO airline (airline_id, airline_code, airline_name, airline_country, created_at, updated_at)

2

VALUES

3

(102, NULL, 'AirEasy', 'France', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP),

4

(103, NULL, 'FlyHigh', 'Brazil', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP),

5

(104, NULL, 'FlyFly', 'Poland', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);

Data Output

Messages

Notifications

SQL

Show

	airline_id [PK] integer	airline_code character varying (30)	airline_name character varying (50)	airline_country character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone
1	101	[null]	KazAir	Turkey	2026-09-27 17:08:48.291306	2026-09-27 17:09:49.936158
2	102	[null]	AirEasy	France	2026-09-27 17:11:11.289873	2026-09-27 17:11:11.289873
3	103	[null]	FlyHigh	Brazil	2026-09-27 17:11:11.289873	2026-09-27 17:11:11.289873
4	104	[null]	FlyFly	Poland	2026-09-27 17:11:11.289873	2026-09-27 17:11:11.289873

By using INSERT INTO and VALUES syntaxes

5. Delete all flights whose arrival in 2024 year:

Query

Query History

1

DELETE FROM flights

2

WHERE sch_arrival_time >= '2024-01-01'

3

AND sch_arrival_time < '2025-01-01';

Scratch Pad

Data Output

Messages

Notifications

flight_id

[PK] integer

sch_departure_time

timestamp without time zone

sch_arrival_time

timestamp without time zone

departing_airport_id

integer

arriving_airport_id

integer

departing_gate

text

arriving_gate

character varying (50)

airline_id

integer

act_departure_time

timestamp without time zone

act_arrival_time

timestamp without time zone

cr

ti

Here I made a mistake, took a screenshot after running the query, should have shown before and after.

6. Increase the price of all tickets in booking table for flights by 15%:

Before:

Query

Query History

1

SELECT * FROM booking

Data Output

Messages

Notifications

After:

Query

Query History

1

UPDATE booking

2

SET ticket_price = ticket_price * 1.15,

3

updated_at = CURRENT_TIMESTAMP;

Data Output

Messages

Notifications

(ticket_price went from 300 to 345)

7. Delete all tickets where price is less than 10000:

Before:

Query

Query History

1

SELECT * FROM booking

After:

9. Delete all baggage check records created before June 1, 2023, whose result is still 'Not checked':

Query

Query History

1

SELECT * FROM baggage_check

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑

📦

📥

📶

SQL

	baggage_check_id [PK] integer	check_result character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone	booking_id integer	passenger_id integer
1	2	checked	2026-09-27 17:34:09.064634	2026-09-27 17:34:09.064634	2	2

After:

Query

Query History

1

DELETE FROM baggage_check

2

WHERE created_at < '2023-06-01'

3

AND check_result = 'Not checked';

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑️

📦

📥

📶

SQL

	baggage_check_id [PK] integer	check_result character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone	booking_id integer	passenger_id integer
1	2	checked	2026-09-27 17:34:09.064634	2026-09-27 17:34:09.064634	2	2

(since there are no older baggage checks, nothing is changed)

10. Delete all entries from the airport table where state is missing (NULL) and the city is either 'Mlawe' or 'Kepuh'

Query		Query History					
1	DELETE FROM airport						
2	WHERE state IS NULL						
3	AND (city = 'Mlawe' OR city = 'Kepuh');						
Data Output							
Showing rows: 1 to 200							
	airport_id [PK] integer	airport_name character varying (50)	country character varying (50)	state character varying (50)	city character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone
1	5001	Airport 1	France	[null]	Paris	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747
2	5002	Airport 2	France	[null]	Paris	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747
3	5003	Airport 3	Kazakhstan	[null]	Paris	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747
4	5004	Airport 4	France	[null]	Paris	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747
5	5005	Airport 5	Kazakhstan	[null]	Almaty	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747
6	5006	Airport 6	France	[null]	Paris	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747
7	5007	Airport 7	France	[null]	Almaty	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747
8	5008	Airport 8	Kazakhstan	[null]	Almaty	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747
9	5009	Airport 9	France	[null]	Paris	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747
10	5010	Airport 10	Kazakhstan	[null]	Almaty	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747
11	5011	Airport 11	France	[null]	Almaty	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747
12	5012	Airport 12	Kazakhstan	[null]	Almaty	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747
13	5013	Airport 13	Kazakhstan	[null]	Almaty	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747
14	5014	Airport 14	France	[null]	Almaty	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747
15	5015	Airport 15	France	[null]	Almaty	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747
16	5016	Airport 16	Kazakhstan	[null]	Paris	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747
17	5017	Airport 17	Kazakhstan	[null]	Paris	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747
18	5018	Airport 18	Kazakhstan	[null]	Paris	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747
19	5019	Airport 19	Kazakhstan	[null]	Almaty	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747
20	5020	Airport 20	France	[null]	Almaty	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747
21	5021	Airport 21	Kazakhstan	[null]	Paris	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747
22	5022	Airport 22	France	[null]	Almaty	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747
23	5023	Airport 23	France	[null]	Almaty	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747
24	5024	Airport 24	France	[null]	Paris	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747
25	5025	Airport 25	Kazakhstan	[null]	Paris	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747

11. Insert a new unchecked baggage inspection record and immediately return its newly generated baggage_check_id and created_at timestamp:

Query		Query History					
1	SELECT * FROM baggage_check						
Data Output							
	baggage_check_id [PK] integer	check_result character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone	booking_id integer	passenger_id integer	
1	2	checked	2026-09-27 17:34:09.064634	2026-09-27 17:34:09.064634	2	2	

After

Query

Query History

1

INSERT INTO baggage_check (baggage_check_id, check_result, created_at, updated_at, booking_id, passenger_id)

2

VALUES (9001, 'Not checked', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP, 2, 1)

3

RETURNING baggage_check_id, created_at;

Data Output

Messages

Notifications

SQL

	baggage_check_id [PK] integer	check_result character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone	booking_id integer	passenger_id integer
1	2	checked	2026-09-27 17:34:09.064634	2026-09-27 17:34:09.064634	2	2
2	9001	Not checked	2026-09-27 17:41:04.694352	2026-09-27 17:41:04.694352	2	1

12.Convert all country names in the airline table to uppercase to ensure data consistency:

Query

Query History

1

UPDATE airline

2

SET airline_country = UPPER(airline_country);

Data Output

Messages

Notifications

13.Update both the name and country for the airline with airline_id = 5. Set the name to 'Global Airways' and the country to 'United Kingdom', while updating update_at to the current date

Query

Query History

1

2

SELECT * FROM airline

Data Output

Messages

Notifications

After

Query

Query History

1

UPDATE airline

2

SET airline_name = 'Global Airways',

3

airline_country = 'United Kingdom',

4

updated_at = CURRENT_DATE

5

WHERE airline_id = 5;

Data Output

Messages

Notifications

SQL

	airline_id [PK] integer	airline_code character varying (30)	airline_name character varying (50)	airline_country character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone
1	101	UNK	KazAir	TURKEY	2026-09-27 17:08:48.291306	2026-09-27 17:27:46.900552
2	102	UNK	AirEasy	FRANCE	2026-09-27 17:11:11.289873	2026-09-27 17:27:46.900552
3	103	UNK	FlyHigh	BRAZIL	2026-09-27 17:11:11.289873	2026-09-27 17:27:46.900552
4	5	UNK	Global Airways	United Kingdom	2026-09-27 17:11:11.289873	2026-09-27 00:00:00

14. Set the state field to 'Capital District' for airports located in specific cities: 'Astana', 'London', and 'Tokyo'

Query

Query History

1

UPDATE airport

2

SET state = 'Capital District'

3

WHERE city = 'Astana' OR city = 'London' OR city = 'Tokyo';

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 200

	airport_id [PK] integer	airport_name character varying (50)	country character varying (50)	state character varying (50)	city character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone
93	5004	Airport 4	France	Capital District	Paris	2026-09-27 17:06:39.634747	2026-09-27 17:06:39.634747

15. Mark all baggage check records (baggage_check) created in March 2024 with a status of 'Not checked' as completed ('Checked')

Query

Query History

1

UPDATE baggage_check

2

SET check_result = 'Checked', updated_at = CURRENT_TIMESTAMP

3

WHERE check_result = 'Not checked'

4

AND created_at >= '2024-03-01'

5

AND created_at < '2024-04-01';

Data Output

Messages

Notifications

SQL

	baggage_check_id [PK] integer	check_result character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone	booking_id integer	passenger_id integer
1	2	checked	2026-09-27 17:34:09.064634	2026-09-27 17:34:09.064634	2	2
2	9001	Not checked	2026-09-27 17:41:04.694352	2026-09-27 17:41:04.694352	2	1